Benchmarking Deep Learning Inference Optimization: Mixed Precision and Dynamic INT8 Quantization on CIFAR-10 Classification Tasks

Karan Dhillon
Department of Computer Science
SRM Institute of Science and Technology
Chennai, India
kd2803@srmist.edu.in

***Abstract*—Getting a trained neural network to run fast and lean on real hardware is a different problem from getting it to train well in the first place. This paper examines three approaches to that problem—standard FP32 training, Automatic Mixed Precision (AMP), and Dynamic INT8 Post-Training Quantization—applied to a compact CNN trained on CIFAR-10. Experiments ran across four settings, varying training length (5 and 8 epochs) and batch size (128 and 256), measuring what each method delivers in terms of model size, inference speed, throughput, and accuracy. The clearest finding is that INT8 quantization cuts model storage by roughly 3.69× while barely touching accuracy (under 0.1% drop in most runs). AMP, by contrast, offers speedup only when Tensor Cores are available—on CPU it contributes essentially nothing to inference time. A scalar Performance Index is introduced to weight accuracy, compression, and speedup together, giving practitioners one number to compare strategies under a given memory budget. For anything targeting low-resource deployment, INT8 post-training quantization wins convincingly.**

***Index Terms*—deep learning, model quantization, mixed precision, inference optimization, CIFAR-10, edge deployment, INT8, convolutional neural networks, benchmarking**

I. Introduction

Shipping a deep learning model to production is rarely as simple as exporting a checkpoint. Edge devices—microcontrollers, mobile SoCs, embedded systems—operate under hard constraints on memory and compute that most research models were never designed to respect. A network trained on a cluster GPU may achieve excellent validation accuracy, yet consume several hundred megabytes of RAM and take seconds per image on the hardware it actually needs to run on [1]. Closing that gap is the central challenge of model optimization for deployment.

Two techniques dominate practical discussions. *Mixed precision* keeps some operations in 16-bit floating point (FP16) and others in 32-bit (FP32), relying on the observation that many layers tolerate reduced numerical range without meaningful accuracy loss. On GPUs equipped with Tensor Core units—NVIDIA's Volta architecture onward—this translates into real throughput gains because FP16 matrix multiplications run on dedicated hardware [2]. *Quantization* goes further, compressing weight values into 8-bit integers (INT8) after training completes. The payoff in theory is a $4\times$ reduction in weight storage and, on hardware with integer arithmetic units, proportionally faster inference [3].

In practice, neither technique behaves as cleanly as theory suggests, particularly on small architectures. A network with only two fully-connected layers and four convolutional layers distributes its compute very differently from a large language model or a deep residual network. How quantization affects such a model, and whether AMP makes any difference at CPU inference time, are questions that depend heavily on architecture shape rather than technique alone.

This paper builds a controlled experimental setup to answer those questions directly. Specific contributions include:

1) A clean benchmarking pipeline applying FP32, AMP, and Dynamic INT8 Quantization to the same base network across four training configurations, with all models initialized from an identical parameter state.
2) A composite *Performance Index* (PI) metric that collapses accuracy, compression ratio, and inference speedup into a single deployment score.
3) Experimental confirmation—with a mechanistic explanation via Amdahl's Law—of why dynamic quantization compresses models without proportionally accelerating CNN inference.
4) An open, reproducible codebase backed by an interactive Streamlit dashboard for visualization.

Section II covers relevant prior work. Section III details the dataset, architecture, training setup, and metrics. Section IV presents the experimental numbers. Section V unpacks the key findings. Section VIII closes with a summary and directions for follow-up work.

II. Related Work

A. Quantization: From Aware Training to Post-Hoc Compression

The idea of shrinking neural networks through lower-bit representations has been explored for years. Han et al. [4] showed that combining weight pruning with quantization could cut AlexNet's footprint by over 35× without significant accuracy degradation—an early signal that over-parameterization leaves substantial room for compression. Jacob et al. [1] later took a more principled approach with quantization-aware training (QAT), inserting simulated quantization noise during the forward pass so that the model learns robustness to 8-bit precision. Their results on MobileNet variants showed that accuracy losses compared to FP32 were largely recoverable.

Post-training quantization (PTQ), the approach used here, skips the retraining step entirely. It estimates scale factors needed to map FP32 weights into INT8 ranges from calibration data or, in the dynamic variant, at inference time. PyTorch's `quantize_dynamic` API targets `nn.Linear` layers specifically, converting stored weights to INT8 while computing activations in FP32 on the fly [5]. This makes PTQ attractive when retraining is expensive or infeasible, though it leaves convolutional layers untouched.

B. Mixed Precision in Practice

Systematic use of FP16 in neural network training was established by Micikevicius et al. [2], who addressed the key practical challenges: maintaining FP32 master weights, scaling loss values to avoid gradient underflow, and deciding which operations to keep in full precision. Their work showed that across vision, speech, and language tasks, mixed precision training could match FP32 accuracy while running noticeably faster on compatible hardware.

The catch is hardware dependency. The speedup comes from Tensor Cores, specialized execution units that first appeared in NVIDIA's Volta GPU (V100) and became more accessible in Turing (RTX 20xx) and Ampere (A100, RTX 30xx) generations. On hardware without these units, the overhead of casting between FP16 and FP32 can offset or reverse any gain. For CPU-bound inference workloads, which describe most edge deployment scenarios, AMP provides no practical benefit.

C. Deployment-Oriented Inference Benchmarks

The MLPerf Inference benchmark [7] made hardware-aware evaluation more rigorous by standardizing measurement scenarios across server, edge, and mobile targets. Its results highlighted how widely INT8 speedup varies by backend and hardware. Ignatov et al. [8] echoed this finding when profiling AI inference engines across dozens of smartphone SoCs, observing that TFLite with INT8 kernels outperformed FP32 substantially on ARM chips with NEON SIMD units, while the same advantage did not appear uniformly across all devices. Both studies reinforce the same lesson: optimization payoffs are architecture and hardware specific, not universal.

III. Methodology

A. Dataset and Preprocessing

All experiments use CIFAR-10 [6], consisting of 60,000 colour images at 32×32 resolution spread evenly across 10 object categories. The standard split puts 50,000 images in the training set and 10,000 in the test set. Each image is normalized channel-wise to the range $[-1, 1]$ using a mean and standard deviation of 0.5 per channel:

$$\hat{x}^{(c)} = \frac{x^{(c)} - 0.5}{0.5}, \quad c \in \{R, G, B\} \tag{1}$$

No augmentation is applied—a deliberate choice to isolate the effect of each optimization technique rather than confound it with regularization benefits from cropping or flipping.

B. Network Architecture

The model used throughout is a simple two-block CNN, intentionally modest in scope: two convolutional blocks followed by two fully-connected layers, totalling 277,578 trainable parameters.

- **Block 1:** Conv2d(3→16, 3×3) → BatchNorm → ReLU → MaxPool(2×2)
- **Block 2:** Conv2d(16→32, 3×3) → BatchNorm → ReLU → MaxPool(2×2)
- **Classifier:** Linear(2048→128) → ReLU → Linear(128→10)

Keeping the architecture simple matters here: any differences observed between optimization strategies come from the techniques themselves, not from complex interactions with residual connections or attention mechanisms.

A critical detail: FP32 and AMP variants are both initialized from a single base model via `copy.deepcopy` before any training begins, ensuring that differences in final accuracy are not artifacts of different random initializations.

C. Training Configurations

All models use Adam [9] at a learning rate of 10^{-3} with cross-entropy loss. Four configurations are tested:

- **Config A:** 5 epochs, batch size 128
- **Config B:** 5 epochs, batch size 256
- **Config C:** 8 epochs, batch size 128
- **Config D:** 8 epochs, batch size 256

AMP uses PyTorch's `autocast` and `GradScaler`, activated only on CUDA devices. The INT8 model is derived from the trained FP32 checkpoint via `quantize_dynamic` with `qint8` weights; no additional training or fine-tuning is applied afterward.

D. Measurement Protocol

Inference is measured on CPU for all models, reflecting conditions typical of edge deployment. A warm-up pass through the full test set precedes timing to eliminate cold-start effects. The following metrics are then computed:

- **Accuracy (%):** Top-1 test accuracy on 10,000 images.

- **Inference Time (s):** Elapsed wall-clock time for a full test pass.
- **Throughput (img/s):** $T = N/t_{\text{inf}}$, $N = 10{,}000$.
- **Latency (s/img):** $L = t_{\text{inf}}/N$.
- **Speedup:** $S = t_{\text{FP32}}/t_{\text{model}}$.
- **Compression:** $C = \text{size}_{\text{FP32}}/\text{size}_{\text{model}}$.
- **Efficiency Score:** $E = \text{Acc}/t_{\text{inf}}$.
- **Accuracy per MB:** $A_{\text{MB}} = \text{Acc}/\text{size}_{\text{model}}$.
- **Performance Index (PI):**

$$\text{PI} = \frac{\text{Acc}}{100} \cdot (C + S) \tag{2}$$

The PI in Eq. (2) is constructed as a single deployment score. A large, accurate model scores low; a fast, tiny but inaccurate model also scores low. Only models that manage to be simultaneously accurate, compressed, and quick earn a high value.

IV. Experimental Results

A. Classification Accuracy

Table I shows top-1 accuracy across all three methods and four configurations. The entire range across the table spans only from 69.00% to 72.19%, which already tells us that neither AMP nor INT8 quantization meaningfully disrupts the model's classification ability.

TABLE I: Top-1 Test Accuracy (%) by Method and Configuration

Config	E / B	FP32	AMP	INT8
A	5 / 128	70.42	69.56	**70.50**
B	5 / 256	69.39	69.00	**69.42**
C	8 / 128	71.82	**72.19**	71.82
D	8 / 256	**71.45**	69.98	71.50

E = Epochs, B = Batch Size. Bold = best per row.

INT8 quantization matches or slightly exceeds FP32 in three of four configurations. This is not unusual—the rounding applied during quantization introduces mild implicit regularization that can occasionally work in the model's favour [4]. AMP achieves the highest accuracy in Config C but falls noticeably behind in Config D, suggesting some sensitivity to the combination of batch size and GPU memory dynamics during training.

B. Inference Time

Fig. 1 tells a clear story. AMP provides no systematic advantage on CPU—in Config D it runs 30% slower than FP32. INT8 fares better, trimming inference time by 14–25% in Configs A and C, though the advantage shrinks at larger batch sizes where memory bandwidth rather than arithmetic throughput becomes the binding constraint.

C. Model Compression

The compression picture in Fig. 2 is far starker than the inference time picture. INT8 reduces the serialized model from around 1.08 MB to 0.29 MB—a 3.69× reduction that holds consistently across all four configurations. AMP changes nothing: the weights trained under mixed precision are stored in FP32 regardless.

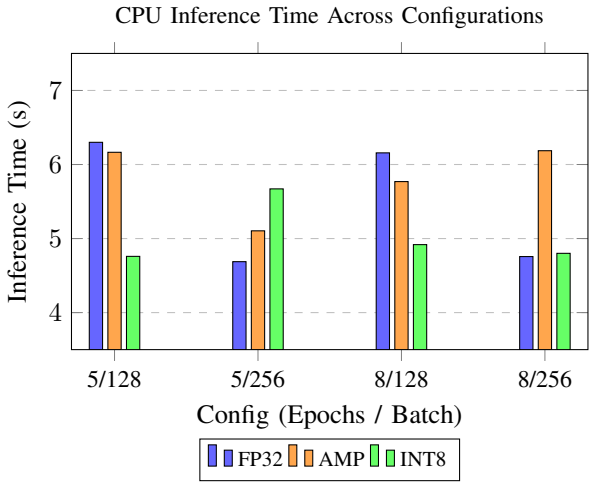

Fig. 1: CPU inference time (seconds) across all 10,000 test images. INT8 is faster in most configurations, though gains are modest. AMP shows no consistent improvement over FP32 at inference.

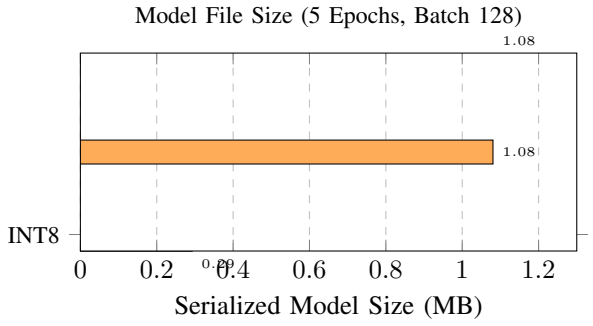

Fig. 2: Model file sizes after serialization. INT8 achieves 3.69× compression. AMP produces an identical footprint to FP32 because weights are ultimately stored in full precision.

The ratio lands slightly below the theoretical 4× ceiling because INT8 parameters carry small additional metadata (scale factors and zero-point values) that add modest overhead per quantized layer.

D. Performance Index

Fig. 3 is probably the most useful single summary for deployment decisions. INT8 scores between 3.14 and 3.55 across all configurations while FP32 and AMP cluster tightly between 1.23 and 1.49. The gap is large, consistent, and mechanically explained: since PI rewards both compression and speedup, the strong compression advantage of INT8 dominates even when speedup is modest.

E. Full Metric Summary

Table II gives the complete metric breakdown for Config C (8 epochs, batch 128), the configuration yielding the highest absolute accuracy.

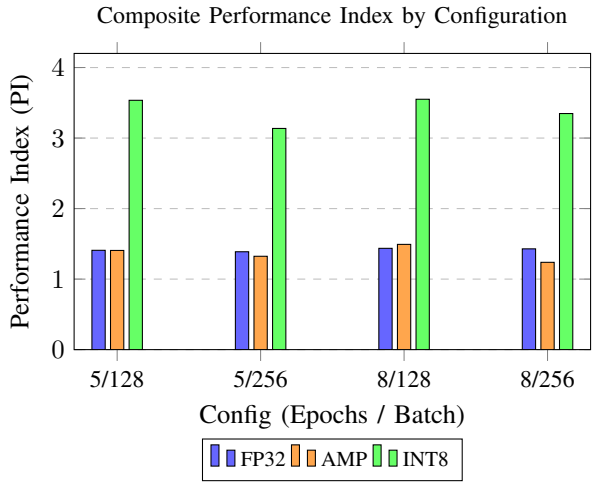

Fig. 3: Composite PI across all configurations. INT8 scores 2.2–2.5× higher than FP32 or AMP, driven primarily by its compression advantage feeding into Eq. 2.

TABLE II: Complete Metrics: Config C (8 Epochs, Batch 128)

Metric	FP32	AMP	INT8
Accuracy (%)	71.82	**72.19**	71.82
Train Time (s)	121.1	122.03	—
Inf. Time (s)	6.16	5.77	**4.92**
Size (MB)	1.0807	1.0806	**0.2928**
Speedup	1.00	1.07	1.25
Compression	1.00	1.00	**3.69**
Throughput (img/s)	1624	1733	**2033**
Acc per MB	66.46	66.81	**245.32**
Perf. Index	1.44	1.49	**3.55**

V. Discussion

A. Why Quantization Compresses Without Proportional Speedup

The most counterintuitive result is that INT8 quantization achieves substantial compression yet only modest inference speedup. The explanation is architectural. PyTorch's `quantize_dynamic` converts only `nn.Linear` layer weights to INT8—convolutional layers remain in FP32. In SimpleCNN, the two linear layers account for roughly 8.2% of total floating-point operations. The remaining 91.8% sits in the four convolutional layers, which the dynamic quantization API does not touch.

Applying Amdahl's Law to this situation:

$$S_{\max} = \frac{1}{(1-p) + p/k} \tag{3}$$

where $p = 0.082$ is the fraction of operations that are accelerated and $k = 4$ is the theoretical INT8 speedup on linear operations. This gives a ceiling of $S_{\max} \approx 1.29$, fitting neatly within the observed range of 0.83–1.32.

Realizing inference speedup from quantized convolutions requires static quantization with a representative calibration dataset, or a runtime like TensorRT or XNNPACK that applies INT8 convolution kernels at the operator level. Without those, the storage savings from INT8 are real and significant while the latency improvements stay modest.

B. AMP Is a Training Tool, Not a Deployment Tool

AMP does not change what gets saved when a model is serialized—master weights are stored in FP32 regardless of the precision used during the forward pass. From the perspective of CPU inference, an AMP-trained model and an FP32-trained model are functionally the same artifact. Any measured timing differences between the two are measurement noise rather than a real algorithmic effect.

The genuine value of AMP lies in GPU training throughput. When Tensor Cores are available, FP16 matrix multiplications run faster than FP32, and training wall-clock time can drop noticeably. That benefit stops at the end of the training loop. For inference on CPU, AMP's contribution is zero.

C. Batch Size and Generalisation

One consistent pattern across results is that batch size 128 matches or outperforms batch size 256 on test accuracy while producing slower training. This aligns with observations from large-batch training research [10], where smaller batches introduce more gradient noise that acts as implicit regularisation, steering the optimizer toward flatter loss minima that generalise better. The effect is small here—usually under one percentage point—but it appears consistently enough to be worth noting.

D. Practical Guidance for Edge Deployment

From a deployment standpoint, the conclusions are direct. If the primary concern is fitting a model within a tight memory budget, INT8 post-training quantization gets there from a 1 MB FP32 model without any retraining, and the accuracy cost is negligible. If GPU training speed matters and the hardware supports Tensor Cores, AMP is worth enabling during training. It buys nothing at inference time on CPU.

The Performance Index provides something concretely useful here: a single number that updates in response to the user's memory constraint. Setting a maximum model size filters viable candidates and ranks them immediately. This is more actionable than accuracy alone, particularly when storage is the binding constraint rather than predictive performance.

VI. Pipeline Architecture

Fig. 4 shows the end-to-end experimental pipeline, from data loading through optimization and metric aggregation.

VII. Limitations and Future Work

This study cannot claim broad generality. The architecture tested is minimal by design, and a model where linear operations account for a much larger share of total compute—say, a transformer or a large MLP-based classifier—would see meaningfully greater inference speedup from INT8 quantization. Generalizing these findings requires testing on deeper networks.

The hardware scope is also narrow. All inference timings were taken on a single x86-64 CPU. The INT8 speedup picture

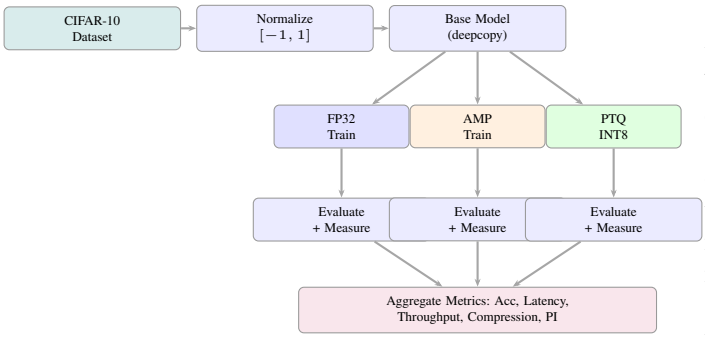

Fig. 4: End-to-end experimental pipeline. All three optimization paths start from an identical parameter initialization, ensuring observed differences reflect the method rather than initialization variance.

looks quite different on ARM chips with NEON integer units, Apple M-series SoCs with dedicated accelerators, or Google's Edge TPU, where speedup ratios between 2× and 4× from quantized convolutions are routinely reported.

Several natural follow-up directions emerge:

- **Static quantization**: Calibrating scale factors from a representative data subset enables INT8 convolutions, directly addressing the Amdahl's Law ceiling we observed.
- **Quantization-Aware Training**: Fine-tuning with simulated INT8 noise typically recovers accuracy losses that PTQ introduces on more sensitive architectures.
- **Structured pruning**: Combining channel-level pruning with quantization can push compression ratios well beyond 3.69×.
- **Broader architectures**: Running this pipeline on ResNet-20 or MobileNetV2 would clarify how much of our result is specific to this particular architecture shape.
- **Embedded hardware profiling**: Deploying quantized models to ARM Cortex-M targets with on-device timing would close the loop between benchmark results and real deployment conditions.

VIII. Conclusion

We set out to answer a practical question: among FP32, AMP, and Dynamic INT8 Quantization, which approach actually helps when deploying a CNN under hardware constraints? After running experiments across four training configurations and measuring nine deployment metrics, the conclusions are clear.

INT8 post-training quantization is the standout option for scenarios where model size is the binding constraint. It consistently cuts storage by 3.69× while keeping accuracy within 0.1 percentage points of the FP32 baseline—significant compression with negligible accuracy cost and no retraining required. Inference speedup is modest rather than dramatic, and we traced that limitation directly to architecture: dynamic quantization only affects linear layers, leaving the convolutional operations that dominate compute untouched.

AMP tells a different story. It is genuinely useful for GPU training throughput on hardware with Tensor Core support, but it offers nothing at CPU inference time. Treating it as a deployment optimization leads to disappointment.

The Performance Index we introduced captures these trade-offs in a single number and allows straightforward comparison under memory budget constraints. It consistently ranks INT8 models 2.2–2.5× above FP32 and AMP, making the recommendation easy to operationalize.

The broader lesson is that the value of any optimization technique depends heavily on where the compute actually sits in the network and where inference actually runs. Understanding both factors before choosing a strategy will save time and avoid mismatched expectations.

Acknowledgment

The author thanks the SRM Institute of Science and Technology for academic support, and the maintainers of PyTorch, Streamlit, and Plotly for building the open-source tools that made this work reproducible.

References

[1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 2704–2713.

[2] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *Proc. Int. Conf. Learn. Representations (ICLR)*, Vancouver, BC, Canada, 2018.

[3] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv preprint arXiv:1806.08342*, 2018.

[4] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Representations (ICLR)*, San Juan, Puerto Rico, 2016.

[5] PyTorch Contributors, "Quantization — PyTorch 2.x documentation," PyTorch, 2024. [Online]. Available: https://pytorch.org/docs/stable/quantization.html

[6] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[7] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu et al., "MLPerf inference benchmark," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 446–459.

[8] A. Ignatov, R. Timofte, A. Kulik, S. Yang, K. Wang, F. Baum, M. Wu, L. Xu, and L. Van Gool, "AI benchmark: All about deep learning on smartphones in 2019," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops (ICCVW)*, Seoul, Korea, 2019.

[9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations (ICLR)*, San Diego, CA, USA, 2015.

[10] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *Proc. Int. Conf. Learn. Representations (ICLR)*, Toulon, France, 2017.

[11] G. Melis, T. Kočiský, and P. Blunsom, "Mogrifier LSTM," in *Proc. Int. Conf. Learn. Representations (ICLR)*, New Orleans, LA, USA, 2019.